

```
    }
}
```

One use of delegates is to provide predicates for methods. A predicate is a method that takes an argument, runs a test over it, and returns the result of that test. These tests tend to be small enough that defining a separate method in a class for them is too time-consuming. For this purpose, we have anonymous delegates and (as of C# 3.0) lambda expressions to fulfill that role. Note that these delegates have type `Func<T1, ..., Tn, R>` where T_i is the type of the parameters and R is the type of the return value. In the case of the void delegates, you use `Func<T>`.

```
public static class AnonDelegateTest
{
    public static void Test()
    {
        Func<int, bool> pos1 = delegate(int x) { return x
> 0; };
        Func<int, bool> pos2 = x => x > 0;
        Func<int, bool> pos3 = (x) => { return x > 0; };
        Console.WriteLine(pos1(1));
        Console.WriteLine(pos2(-1));
        Console.WriteLine(pos3(0));
    }
}
```

Note that `Func` resides in `System.Core.dll` rather than `mscorlib.dll`. The latter is referenced by default so you need to add in `System.Core.dll` as a new reference to your project if you want to use the `Func` class.

Delegates are extremely handy as ways to parameterise functions by behaviour (e.g., send a sort function to a method or class) or for our purposes, providing callbacks for objects registering for an event. From our example above, a delegate corresponds to the function that the program wants the `Button` object to invoke when it is pressed. Thus, the process of registering for the event amounts to sending the `Button` object a delegate object to invoke.

6.2 Events

From our description above, we could implement events using multicast delegates without much problem. However, C# provides direct support for events as class members.

```
public class EventBasics
```